

ROLPLAY

Unified Analytics Dashboard

End-to-End Engineering Documentation



One multi-tenant dashboard serving every Rolplay client, whatever backend their data lives in — architecture, data pipelines, schemas, API reference, and the AI dashboard builder.

Document	End-to-end technical documentation
Repository	Rolplay_Dashboard_Project
Branch / commit	main @ 008dedf
Generated	29 July 2026
Audience	Engineering, technical leadership, onboarding
Classification	Internal — confidential

Contents

#	Section	Covers
1	What we are building, and why	Problem, goals, non-negotiable principles
2	System architecture	Layers, request lifecycle, repository layout
3	Multi-tenancy	Three resolution mechanisms, dynamic tenants, caching
4	Authentication and authorization	Postgres auth DB, JWT, guards, partial access
5	Data pipelines (connectors)	Every upstream, how they differ, module gating
6	Data schemas	Postgres auth DB, tenant config, MySQL analytics, AI tables
7	API reference	All Next.js routes and the FastAPI AI service
8	TypeScript domain contracts	Shared types the UI and routes agree on
9	AI dashboard builder	Agent pipeline, job phases, guided configuration flow
10	Frontend structure	Routes, components, client state, dynamic rendering
11	Localization and demo mode	ES/EN dictionary, seeded demo engine, scoping
12	Technology stack	Versions, tooling, test suite
13	Deployment and configuration	Environment variables, targets, migrations
14	Known issues and open items	Verified discrepancies and what is still pending

On accuracy. Every fact in this document was read out of the repository at the commit on the cover page. Where the code and an older migration disagree, both are stated and the live behaviour is identified — see section 14 rather than assuming the migration is authoritative.

1. What we are building, and why

1.1 The problem

Rolplay sells AI-driven commercial simulation and coaching. Over time each client was delivered a **bespoke dashboard**, wired directly to whichever backend happened to hold that client's data. The result was roughly a dozen separate frontends with duplicated logic, no shared design language, and a per-client engineering cost every time a metric or a branding tweak was requested. Onboarding a new client meant building a new dashboard.

The upstreams genuinely differ — they are not one API behind a wall. Some clients are served by an action-dispatch bridge over MySQL, some by pre-existing Flask REST services, some by a knowledge-base product with its own API and token, and one by a raw read-only SQL endpoint. Any unification had to absorb that variety rather than pretend it away.

1.2 What this project is

One multi-tenant Next.js dashboard that serves every client. A user signs in with their work email; the platform resolves which client they belong to, which upstream holds their data, and which modules they actually have — then renders only those. No per-client frontend, no per-client deploy.

- **A single codebase, many tenants.** Tenant identity is resolved at request time from the session, never from a build-time constant.
- **Connector-per-upstream, one contract.** Each backend gets an adapter that normalizes into shared types, so pages are written once against those types.
- **Capability-driven rendering.** The navigation and pages are generated from what the tenant demonstrably has, not from a hardcoded per-client layout.
- **Self-service onboarding.** An admin registers a client through a wizard, or the AI builder discovers and generates the dashboard, with no code change.

1.3 Product surface

Module	What it measures	Where the data comes from
Simulator	Practice-conversation sessions, scores, pass rate	Pharma bridge, coach_app MySQL, or rolplay.app SQL
Master Coach	Coaching sessions and derived coaching insights	Separate activity IDs (kpi tenants) or a coaching lens on sim rows
Certifier Coach	Formal certification attempts, pass/fail	Only where a genuinely separate certification source is confirmed
LMS	Course enrollments, completion rate, quiz scores	LearnWorlds API (lib/lms-learnworlds.ts), per-tenant credentials
Second Brain	Knowledge-base org, members, documents	Dedicated Second Brain REST API and token — never the SQL bridge
Conversational / Objections	Objection handling and model answers	Bridge objections endpoint, capability-gated
Business Lines	Per-business-line performance	Members-tag catalog, capability-gated
Organization	Roster: members, admins, supervisors	Bridge org endpoints, capability-gated

1.4 Principles the code actually enforces

These are not aspirations; they are invariants with tests and code comments defending them, and they explain most of the design decisions in this document.

1. **Never invent data.** A metric with no verified upstream returns empty or null — never zero-as-a-number and never another module's figures. A tenant without Coach Maestro must not see its Simulator numbers relabelled "Coach Maestro"; a tenant without an LMS must not see session counts labelled "courses". Nullable means "no data upstream", zero means "measured zero".

2. **Capability before render.** Contracted intent and verified data are separate facts. A module renders only when the tenant has it AND it has data.
3. **Tenant isolation by construction.** `customer_id` is re-resolved at login rather than trusted from the user row, so a stale value cannot leak another client's data. Second Brain deliberately has no shared fallback org.
4. **Fail honestly.** When an upstream errors, the route surfaces the error. Module discovery is the one deliberate exception: it fails open to the full module set rather than hiding a client's real data behind an outage.
5. **No hardcoded client layouts.** Everything a client sees is derived from tenant config plus live probes, which is what makes the AI builder possible.

2. System architecture

2.1 Layers

Four layers, with a strict rule: **connectors are server-only**. They hold secrets and upstream URLs, so no client component may import them. Pages reach data exclusively through route handlers.

BROWSER	Next.js App Router pages + React client components zustand store (filters, date range) useApi() hooks fetch, cookies: accessToken (HTTP-only) v	
EDGE	middleware.ts -- page-route auth redirects only. Explicitly SKIPS /api/, injects no tenant headers. v	
SERVER	app/api/**/route.ts (runtime: nodejs) 1. getAuthContextFromRequest() -> {userId, email, customerId} 2. useDemoData(email)? -> synthetic payload, done 3. resolveOrgType(email, customerId) -> 'banco' 'pharma' 'rolplay-app' 'analytics' 'none' 4. dispatch to exactly ONE connector 5. buildSuccess(data, meta) -> {success, data, meta} v	
CONNECTORS (server only)	lib/bridge-pharma-analytics.ts lib/bridge-client.ts lib/bridge-banco*.ts lib/bridge-rolplay-app.ts lib/second-brain-api.ts lib/lms-learnworlds.ts v	pharma action bridges (3 kinds) PHP SQL bridge -> coach_app Banco, domain-scoped rolplay.app raw SQL endpoint Second Brain REST + token LearnWorlds LMS API
UPSTREAMS	MySQL (coach_app, rolplay_pro) per-tenant bridge containers Flask REST (exceltis) Second Brain API LearnWorlds API Postgres (auth + tenant config + AI service tables)	

2.2 The request lifecycle, concretely

A GET of the Simulator page's KPI row resolves like this. The ordering is load-bearing: the demo check precedes tenant resolution so a demo session never touches a real upstream, and tenant resolution precedes dispatch so no connector is ever called with an unresolved tenant.

1. Browser requests /api/dashboard/overview?from=..&to=..&solution=simulator with the accessToken cookie.
2. getAuthContextFromRequest() verifies the JWT (HS256, jose) and yields {userId, email, customerId}. No row read, so this is cheap.
3. useDemoData(email) short-circuits for Rolplay-owned domains.
4. resolveOrgType() classifies the caller. For pharma it awaits ensureDynamicTenantsLoaded(), so DB-registered tenants are visible.
5. The matching connector runs. isUnsupportedModule() decides whether this module is real for this tenant, returning empty rather than blended data.
6. buildSuccess() wraps the payload with meta.source, which names the pipeline that served it — the fastest way to diagnose wrong numbers.

Operational tip. meta.source is the debugging entry point. pharma-apotex means the pharma bridge served it; demo means synthetic; fallback means module discovery failed open; lms-not-configured means the tenant has no LMS. If a client reports wrong numbers, read meta.source before reading any code.

2.3 Repository layout

Path	Contents
app/	App Router pages and all api/**/route.ts handlers

Path	Contents
lib/	Connectors, auth, tenant resolution, types, demo engine, i18n
lib/demo/	Seeded deterministic demo data engine and its scoping guard
lib/hooks/	Client data hooks: useApi, useAvailableModules, snapping
components/	Shared UI, charts, shadcn primitives
migrations/	Numbered SQL migrations (001-005)
ai-service/	FastAPI + LangGraph dashboard-builder service (Python)
docs/	Onboarding runbooks, tenant isolation, wireframes
rolplay-bridge.php	The PHP SQL bridge deployed alongside MySQL upstreams

Careful. The dashboard/ directory and anything under .claude/worktrees/ are duplicates of the main tree and are excluded from the vitest config. Do not edit them; changes there do not ship.

3. Multi-tenancy

Three resolution mechanisms coexist because the upstreams identify clients in three incompatible ways. This is deliberate: collapsing them into one scheme would break at least one real client. `lib/org-type.ts` is the single classifier, and it is checked in a fixed priority order.

Org type	Keyed by	Resolved in	Notes
banco	Email domain	<code>lib/org-type.ts + BANCO_EMAIL_DOMAINS</code>	Never <code>customer_id</code> : <code>banco_users</code> has no email column
pharma	Email domain -> tenant key	<code>resolvePharmaTenant(email)</code>	DB map > env map > built-in aliases
rolplay-app	Explicit login -> <code>client_id</code>	<code>resolveRolplayAppAccess()</code>	Domain is unusable: clients share <code>audioweb.com.mx</code>
analytics	<code>customer_id</code> from the JWT	Resolved at login, stamped in token	<code>0</code> = authenticated but unlinked -> empty state
none	—	—	Authenticated with no reachable data source

3.1 Why `customer_id` is re-resolved at login

`app/api/auth/login/route.ts` calls `resolveCustomerIdByEmail()` on every login and stamps the result into the JWT, **deliberately refusing** to reuse the stored `users.customer_id`. If a user is moved between clients upstream, a stored value would be stale, and a stale tenant id is a cross-tenant data leak. Re-resolving costs one bridge call per login and removes that class of bug entirely.

3.2 Dynamic tenants

`TENANT_CONFIG` in `lib/pharma-tenant.ts` starts as a mutable record of hand-verified static defaults, then merges rows from the `pharma_tenants` Postgres table over it. That merge is what lets an admin onboard a client through the UI with no deploy.

- `ensureDynamicTenantsLoaded()` caches in-process for `DYNAMIC_TENANTS_TTL_MS = 30_000`, de-duplicates concurrent loads behind a single in-flight promise, and treats DB failure as non-fatal (static defaults keep serving).
- **Capability flags are OR-ed** with the static values, so a sparse DB row can **enable** a capability but never silently **drop** one that was verified in code.
- `ucids` and `coachActivityIds` fall back to the static values when the DB row omits them — an empty allowlist would widen a tenant's scope, not narrow it.
- Deactivated tenants are removed from `TENANT_CONFIG` via `previouslyLoadedDynamicKeys`, so deactivation takes effect rather than lingering until restart.
- `invalidateDynamicTenantsCache()` is called by every admin write, so changes are visible immediately instead of after the TTL.

Invariant. `resolvePharmaTenant()` and `resolveOrgType()` are the only places that await the dynamic load. Everything downstream reads `TENANT_CONFIG` synchronously. Keep it that way: awaiting deeper in a connector would reintroduce races.

3.3 Branding is keyed separately

`customer_id` collapses to `0` for every non-coach tenant, so it cannot key branding. `brandingTenantKey()` in `lib/db-branding.ts` produces `cust:<id>` when `customer_id > 0` and `domain:<email-domain>` otherwise, with a unique index on `tenant_key` (migration 004).

4. Authentication and authorization

Users live in a **separate PostgreSQL database** (`AUTH_DATABASE_URL`), reached only through `lib/db-auth.ts`. Analytics data never lands there. That separation means a client's analytics upstream can be swapped without touching identity, and an auth outage degrades rather than exposes.

User store	Postgres: users, user_sessions. Pool singleton on <code>global.__authPool</code> ; <code>sslmode</code> stripped from the URL.
Errors	<code>AuthDbError</code> with codes <code>NOT_CONFIGURED</code> , <code>CONNECTION_FAILED</code> , <code>TABLE_MISSING</code> , <code>DUPLICATE_EMAIL</code> , <code>QUERY_FAILED</code> .
Passwords	<code>bcryptjs</code> via <code>lib/password.ts</code> .
Tokens	<code>jose</code> , HS256. Access 8 h (<code>JWT_SECRET</code>), refresh 7 d (<code>REFRESH_SECRET</code>). Claims: <code>user_id</code> , <code>email</code> , <code>customer_id</code> , <code>jti</code> .
Transport	HTTP-only cookies <code>accessToken</code> and <code>refreshToken</code> . No tokens in URLs or <code>localStorage</code> .
Sessions	<code>user_sessions.token_jti</code> records refresh-token identity, so logout can invalidate a specific refresh token.

4.1 Two guards, two jobs

Guard	Applies to	Behaviour
<code>middleware.ts</code>	Page routes only	Redirects unauthenticated users to <code>/auth/login</code> and authenticated users away from <code>login/register</code> . Skips <code>/api/*</code> , <code>/_next/*</code> , <code>/</code> , <code>/privacy</code> , <code>/terms</code> . Injects no auth or tenant headers.
<code>lib/server-auth.ts</code>	Every API route	<code>getAuthContextFromRequest()</code> verifies the cookie and returns the context (accepting <code>customerId === 0</code>). <code>requireAdminFromRequest()</code> additionally loads the user row, because role is not in the JWT .

Consequence. Middleware skipping `/api/*` is intentional, not an oversight: every route handler authenticates itself. The consequence is that **adding a route means adding its guard** — there is no blanket API protection to inherit. Two diagnostic routes are currently unauthenticated by design; see section 14.

4.2 Role model

Roles are `user` and `admin` (CHECK-constrained). Role deliberately lives outside the JWT so that revoking admin takes effect immediately rather than at token expiry; the cost is one extra `findUserById()` per admin request. Only the five `/api/admin/tenants*` handlers are role-gated. `/api/auth/setup` is gated by a shared secret (`SETUP_SECRET`) instead, because it must run before any admin exists.

4.3 Partial access

A user may legitimately have some data sources and not others, so access is not binary. `getAccessStatus()` in `lib/multi-source-auth.ts` backs `GET /api/auth/access-status`, which reports `hasCoachData`, `hasSecondBrainData`, `hasPharmaAccess`, `hasBancoAccess`, `hasRolplayAppAccess` and `hasAnyAccess`. Pages gate on these individually rather than hard-blocking a user who lacks one source.

4.4 Client-side session handling

`components/AuthProvider.tsx` calls `/api/auth/me` on mount. On a 401 it attempts a silent `POST /api/auth/refresh` and retries once, so an expired access token does not bounce an active user to the login screen. `/api/auth/me` falls back to JWT claims when the auth DB is unreachable, keeping a signed-in user signed in through a database blip.

5. Data pipelines (connectors)

Every connector lives in `lib/`, is server-only, and normalizes its upstream into the shared types in section 8. Each API route resolves the org type, then dispatches to exactly one connector — never two, so a payload can always be attributed to a single pipeline.

Connector	File	Upstream and protocol
PHP SQL bridge	bridge-client.ts, data-provider.ts	MySQL coach_app / rolplay_pro_analytics. POST BRIDGE_URL with {sql, params}, header X-Bridge-Key: BRIDGE_SECRET, response {success, data, error}. Scoped by customer_id on every call.
Generic MySQL	db.ts	Same MySQL. Dual-mode: uses the bridge when BRIDGE_URL is set, else a direct mysql2 pool. Direct mode is for local development.
Banco	bridge-banco.ts, bridge-banco-analytics.ts	Scoped by email domain , never customer_id. Score is parsed out of closingretro HTML, normalized to 0-100, pass at >= 60.
Pharma bridges	bridge-pharma-analytics.ts (~1345 lines)	Per-tenant containers on serv.aux-rolplay.com. POST {action, ...params}, optional X-Tenant header, 30 s timeout. Three sub-kinds — below.
rolplay.app	bridge-rolplay-app.ts	Raw read-only SELECT endpoint (ROLPLAY_APP_SQL_URL) over r_* tables. POST {sql}, response {result, data}.
Second Brain	second-brain-api.ts, banco-second-brain.ts	Dedicated REST API plus token. API-only by rule — never read from a SQL bridge, because the SQL schema has no knowledge-base concept.
LMS	lms-learnworlds.ts	LearnWorlds API. OAuth2 client-credentials (or a static token), Lw-Client header on every call. Per-tenant credentials via LMS_<TENANT>_*.
AI builder	app/api/ai/[...path] -> ai-service/	Reverse proxy to the FastAPI service at AI_SERVICE_URL, 120 s timeout.

5.1 The three pharma bridge kinds

All three are "pharma" to the org classifier, but their wire protocols and schemas differ enough to need separate adapters. TenantConfig.kind selects the adapter.

kind	Clients	Shape
sale_exercises	Sanfer, Weser, Adium	Action-dispatch bridge returning raw per-session rows; the adapter aggregates in-process. Requires a fixed ucids allowlist matching the client's own dashboard scope (Sanfer pins an exact 44-ID list).
kpi	Apotex	Same action bridge, different schema. coachActivityIds (Apotex: 8, 9, 10) identifies the real Coach Maestro module; everything else is Simulator.
exceltis_rest	Heineken, M8, Lacoste, Lacoste Asistentes, Chiesi, Labomed	Pre-existing Flask REST endpoints (GET /api/rol_play_sim_extractor, /api/dim_actividades) — not an action bridge at all.

Score normalization is per-client, on purpose

normalizeSimScore() trusts Calificacion when it is <= 100 and otherwise falls back to Puntos_Totales. Clients genuinely scale scores differently; a single global formula produced visibly wrong numbers for at least one client, which is why the per-client branch exists rather than a shared constant.

5.2 Capability gating: the anti-duplication rule

Capability flags on TenantConfig decide which modules return data. The rule they enforce, stated in the code comments of isUnsupportedModule(), is that **a module with no distinct upstream returns empty rather than falling through to the tenant's default view**. Without this, a tenant lacking Coach Maestro would see its Simulator numbers a second time under a "Coach Maestro" heading — the "same data everywhere" failure the connector exists to prevent.

Flag	Gates	Set true when
hasCertification	Certifier Coach	A genuinely separate certification source is confirmed (Sanfer: official platform DB)
hasObjections	Conversational / Objections	objections.demorp6 is confirmed present and working
hasBusinessLines	Business Lines	A members-tag catalog (tag1) is confirmed
hasOrganization	Organization	org.members / org.admins are confirmed
hasTopStats	All-time leaderboard	sim.topstats is confirmed
hasLms	LMS	The tenant has a real LearnWorlds school with credentials configured
hasSimulator	Simulator	The tenant has practice-simulation sessions (default for bridge tenants)
coachActivityIds	Master Coach	A verified activity-ID split exists (kpi kind)

Subtlety. coach is treated differently from lms and second-brain. For sale_exercises and exceltis_rest tenants there is no separate Coach data source, but coaching is a legitimate **different lens** on the same practice sessions — verified against Sanfer's own CoachingPage.tsx, which derives its insights from the same rows. So coach falls through to the tenant's data, whereas lms and second-brain are blocked outright because no course or knowledge-base data exists in a bridge at all.

5.3 Request coalescing

One dashboard page load fires roughly five concurrent requests that each need the tenant's full session history. `_sessionsCache` in `bridge-pharma-analytics.ts` (30 s TTL) de-duplicates them into a single upstream fetch, which is the difference between a fast page and five redundant multi-second bridge calls.

6. Data schemas

Data lives in several databases with different owners and different degrees of writability. Only the Postgres auth DB is owned by this project; the MySQL analytics databases and the platform DB are read-only from here.

Database	Engine	Access	Contents	Writable?
Auth DB (AUTH_DATABASE_URL)	PostgreSQL	lib/db-auth.ts	Identity, sessions, branding, tenant config, AI service tables	Yes
coach_app	MySQL 8	PHP bridge	Coach / simulator / certification analytics	Read-only
rolplay_pro	MySQL	PHP bridge	Banco pipeline	Read-only
rolplay.app platform	MySQL	Raw SQL endpoint	r_* tables	Read-only
Pharma bridges	n/a (HTTP)	Action / REST	No local DDL	Read-only
LearnWorlds	n/a (HTTP)	REST API	Courses, enrollments	Read-only

6.1 Auth DB: identity

Created by migration 002 and app/api/auth/setup/route.ts

```
users
  id          SERIAL PRIMARY KEY
  email       VARCHAR(255) UNIQUE NOT NULL
  password_hash VARCHAR(255) NOT NULL -- bcrypt
  full_name   VARCHAR(255) NOT NULL DEFAULT ''
  company_domain VARCHAR(255) NOT NULL DEFAULT ''
  customer_id INTEGER NOT NULL DEFAULT 0 -- live shape; see section 14
  role        VARCHAR(20) NOT NULL DEFAULT 'user'
              CHECK (role IN ('user', 'admin'))
  is_active   BOOLEAN NOT NULL DEFAULT TRUE
  last_login  TIMESTAMPTZ
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
  updated_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
  INDEX idx_users_email (email), idx_users_customer_id (customer_id)

user_sessions
  id          SERIAL PRIMARY KEY
  user_id     INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE
  token_jti   VARCHAR(255) UNIQUE NOT NULL -- JWT ID of the refresh token
  expires_at  TIMESTAMPTZ NOT NULL
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
  INDEX idx_sessions_user_id, idx_sessions_expires, idx_sessions_jti
```

6.2 Auth DB: tenant configuration

These two tables are what make onboarding a client a data operation rather than a deploy. pharma_tenants rows are merged over the static TENANT_CONFIG at runtime (section 3.2).

```

pharma_tenants                                -- migration 003
  id                                            SERIAL PRIMARY KEY
  tenant_key                                  VARCHAR(100) UNIQUE NOT NULL
  display_name                                VARCHAR(255) NOT NULL
  kind                                         VARCHAR(20) NOT NULL
                                              CHECK (kind IN ('sale_exercises','kpi','exceltis_rest'))
  url                                          TEXT NOT NULL
  x_tenant                                    VARCHAR(100)
  ucids                                       JSONB NOT NULL DEFAULT '[]'
  has_certification                          BOOLEAN NOT NULL DEFAULT FALSE
  has_objections                             BOOLEAN NOT NULL DEFAULT FALSE
  has_business_lines                         BOOLEAN NOT NULL DEFAULT FALSE
  has_organization                          BOOLEAN NOT NULL DEFAULT FALSE
  has_top_stats                             BOOLEAN NOT NULL DEFAULT FALSE
  coach_activity_ids                         JSONB -- 'kpi' kind only
  auth_header_name                           VARCHAR(100)
  auth_header_value                          TEXT
  is_active                                  BOOLEAN NOT NULL DEFAULT TRUE
  created_by                                 INTEGER REFERENCES users(id)
  created_at                                 TIMESTAMPTZ NOT NULL DEFAULT NOW()
  updated_at                                 TIMESTAMPTZ NOT NULL DEFAULT NOW()
  INDEX idx_pharma_tenants_key (tenant_key)

pharma_tenant_domains                          -- migration 003
  id                                            SERIAL PRIMARY KEY
  domain                                       VARCHAR(255) UNIQUE NOT NULL
  tenant_key                                  VARCHAR(100) NOT NULL
                                              REFERENCES pharma_tenants(tenant_key) ON DELETE CASCADE
  created_at                                 TIMESTAMPTZ NOT NULL DEFAULT NOW()
  INDEX idx_pharma_tenant_domains_domain (domain)

```

Note: has_lms and has_simulator are new capability flags added with the LMS pipeline and are not yet in a numbered migration — see section 14.

6.3 Auth DB: branding and integrations

```

branding_settings                             -- setup route; tenant_key added by migration 004
  id                                            SERIAL PRIMARY KEY
  customer_id                                INTEGER UNIQUE NOT NULL -- legacy key, still UNIQUE
  tenant_key                                  TEXT UNIQUE -- 'cust:<id>' or 'domain:<domain>'
  logo_url                                    TEXT
  primary_color                              TEXT NOT NULL DEFAULT '#DC2626'
  secondary_color                            TEXT NOT NULL DEFAULT '#1F2937'
  accent_color                               TEXT NOT NULL DEFAULT '#14B8A6'
  updated_at                                 TIMESTAMPTZ NOT NULL DEFAULT NOW()

tenant_integrations                          -- setup route only, no numbered migration
  id                                            SERIAL PRIMARY KEY
  customer_id                                INTEGER UNIQUE NOT NULL
  second_brain_admin_email                   TEXT
  second_brain_api_token                    TEXT
  updated_at                                 TIMESTAMPTZ NOT NULL DEFAULT NOW()

rolplay_app_domains                          -- migration 005
  domain                                       TEXT PRIMARY KEY -- lowercase email domain
  client_id                                  INTEGER NOT NULL -- r_client.ID on rolplay.app
  display_name                               TEXT
  is_active                                  BOOLEAN NOT NULL DEFAULT TRUE
  created_at                                 TIMESTAMPTZ NOT NULL DEFAULT NOW()
  updated_at                                 TIMESTAMPTZ NOT NULL DEFAULT NOW()
  INDEX rolplay_app_domains_client_idx (client_id)

```

Why one domain is missing. Migration 005 seeds siigo.com -> 29, takeda.com -> 13, besins-healthcare.com -> 14, rowe.com.do -> 25, rowe.com -> 25. audioweb.com.mx is **deliberately excluded**: it is a shared staff domain, so mapping it would hand one client's dashboard to another's staff.

6.4 Auth DB: AI service tables

The FastAPI service reuses the same Postgres instance and creates its own tables (ai-service/app/db.py). If AUTH_DATABASE_URL is unset it runs fully in-memory, which is why the builder works in local development with no database at all.

agent_memory	slug TEXT PK, company TEXT NOT NULL, payload JSONB NOT NULL, updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
dashboard_metadata	slug TEXT PK, company TEXT NOT NULL, config JSONB NOT NULL, version INT NOT NULL DEFAULT 1, published BOOLEAN NOT NULL DEFAULT FALSE, updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
dashboard_versions	id SERIAL PK, slug TEXT NOT NULL, version INT NOT NULL, config JSONB NOT NULL, created_at TIMESTAMPTZ NOT NULL
discovery_logs	id SERIAL PK, slug TEXT NOT NULL, job_id TEXT, payload JSONB NOT NULL, created_at TIMESTAMPTZ NOT NULL
validation_reports	id SERIAL PK, slug TEXT NOT NULL, job_id TEXT, report JSONB NOT NULL, created_at TIMESTAMPTZ NOT NULL

6.5 MySQL coach_app: the analytics core

40 tables; DATABASE_SCHEMA.md documents all of them. admin_user is the customer registry that every customer_id foreign-keys to, and saved_reports is the central fact table. These are the ones the dashboard actually queries.

```
admin_user          -- the tenant/customer registry
  id int unsigned PK AUTO_INCREMENT
  company_name varchar(190), email varchar(190) NOT NULL UNIQUE
  pwd varchar(255) NOT NULL
  access tinyint(1) NOT NULL DEFAULT 0      -- 0 = customer, 1 = superadmin
  enable_api tinyint unsigned NOT NULL DEFAULT 0, api_key varchar(255)
  logo varchar(100) NOT NULL DEFAULT '', homepage_content varchar(2000)
  date datetime NOT NULL

coach_users
  id int unsigned PK, customer_id int unsigned NOT NULL
  user_email varchar(190) NOT NULL, user_pass varchar(190) NOT NULL DEFAULT ''
  user_name varchar(100) NOT NULL DEFAULT ''
  signup tinyint(1) NOT NULL DEFAULT 0      -- 1 = signed up, 0 = pending
  date_added datetime NOT NULL
  FK customer_id -> admin_user(id) ON DELETE CASCADE

saved_reports       -- the core fact table (coach_app variant)
  id int unsigned PK, uid varchar(191) NOT NULL UNIQUE
  usecase_id int unsigned, coach_user_id int unsigned
  interaction_type tinyint(1) NOT NULL
  elevator_pitch text, closingretro text
  usecase_stage_id int unsigned, eval_session_id int unsigned
  usecase_segment_id int unsigned, segment_num tinyint unsigned
  score int DEFAULT 0, passed_flag smallint DEFAULT 0
  date_created datetime NOT NULL
  FK usecase_id -> coach_usecases(id), coach_user_id -> coach_users(id)

coach_usecases      id, base_usecase_id, customer_id, usecase_name varchar(100),
                    added_instructions text, added_content longtext,
                    direct_interaction_type tinyint -- 1 Realtime, 2 Video, 3 Audio
usecases            base templates; interaction_type 1..7, lang, ai_model_id
usecase_stages      ID, coach_usecase_id, sequence, name, content, instructions
usecase_segment     id, usecase_id, title
segment_contents    id, usecase_segment_id, segment_num, min_score,
                    instruction_prompt, evaluation_prompt, template, questions
coach_teams / coach_team_user / coach_managers / coach_manager_users
coach_evaluation_sessions / _segments / _team
```

All coach_app tables are InnoDB, utf8mb4. Migration 001 (MySQL) also defines clients, a MySQL users/user_sessions pair, and adds company_id to five analytics tables that do not exist in the production dump — that migration reflects an earlier design and is not the live multi-tenancy mechanism.

6.6 MySQL rolplay_pro: the Banco pipeline

A different schema with the same table name. This is why the Banco connector is separate code rather than a parameter.

```
banco_users
  ID int(11) PK AUTO_INCREMENT, emp_id int(11) NOT NULL
  name varchar(50) NOT NULL, parent_emp_id int(11) DEFAULT 0
  position varchar(25) NOT NULL, hide_welcome tinyint(1) NOT NULL DEFAULT 0
  -- NO email column -> Banco must be scoped by login domain, not customer_id

saved_reports (rolplay_pro variant -- NOTE: no score / passed_flag columns)
  id int(11) unsigned PK, uid varchar(191) NOT NULL UNIQUE
  usecase_id int(11) unsigned NOT NULL, interaction_type tinyint(1) NOT NULL
  elevator_pitch text, closingretro text          -- score is parsed out of this
  date_created datetime NOT NULL, banco_user_id int(11)

saved_reports_options
  id PK, saved_report_id int unsigned NOT NULL, seq int unsigned NOT NULL
  gen_ques text, ai_ques varchar(255), user_resp text, retro text
  FK saved_report_id -> saved_reports(id) ON DELETE CASCADE
```

Fragility. The rolplay_pro saved_reports has **no** score **or** passed_flag. Scores are extracted from the closingretro HTML and normalized to 0-100 with a pass threshold of 60. That parsing is the single most fragile piece of the Banco pipeline: an upstream change to that HTML changes the numbers. It has dedicated unit tests (lib/__tests__/bridge-banco-analytics.test.ts).

6.7 rolplay.app platform tables

Queried read-only through the raw SQL endpoint; no DDL exists in this repository. Observed tables: r_user (r_id, r_email, r_name), r_simulator (its category column drives module discovery — COACH, SIM, SEGMENT), r_user_session, r_user_session_details, and r_client (whose ID is the client_id in rolplay_app_domains).

7. API reference

All Next.js handlers run on the Node runtime and return the envelope `{success, data, meta}` via `buildSuccess()` / `buildApiError()`. `meta.source` names the pipeline that served the request. Because `middleware.ts` skips `/api/*`, **each handler authenticates itself**.

Auth column: "Session" = valid accessToken cookie (401 otherwise); "ADMIN" = session plus role === 'admin' (403 otherwise); "None" = unauthenticated by design. Required params are marked *.

7.1 Dashboard data

Route	Auth	Returns	Params
GET /api/dashboard/overview	Session	KPI totals, avg score, pass rate, plus the previous period for deltas	from*, to*, solution, usecaseIds
GET /api/dashboard/trends	Session	Score, pass/fail and eval-count series; optional score distribution	from*, to*, solution, granularity, compare, usecaseIds
GET /api/dashboard/results	Session	Evaluation result rows	from*, to*, solution, limit (default 50, max 200), usecaseIds
GET /api/dashboard/usecase-breakdown	Session	Per-usecase / per-simulator aggregation	from*, to*, solution, usecaseIds
GET /api/dashboard/best-performers	Session	Leaderboard; adds all-time stats where hasTopStats	from*, to*, limit (1-5), solution, usecaseIds
GET /api/dashboard/lms	Session	LMS course metrics: enrollments, completion rate, quiz scores, per-course rows	from*, to*
GET /api/dashboard/business-lines	Session	Business-line catalog; empty unless hasBusinessLines	from*, to*
GET /api/dashboard/objections	Session	Objection handling; empty unless hasObjections	from*, to*
GET /api/dashboard/organization	Session	Current-state roster: members, admins, supervisors	none (current state)
GET /api/dashboard/modules	Session	Which modules this tenant has. Fails open to the full set on error	none
GET /api/dashboard/data-bounds	Session	Real data span for snapping the default range; null if unknown	none
GET /api/dashboard/drilldown/[savedReportId]	Session	Single saved-report detail	path id (400 if not numeric)
GET /api/banco	Session	Isolated Banco KPIs plus 20 recent sessions	from, to (default trailing 30 d)
GET /api/second-brain/profile	Session	Second Brain org profile, phone numbers masked	none

7.2 Authentication

Route	Auth	Purpose
POST /api/auth/login	Public	Verify password, re-resolve customer_id, set both cookies
POST /api/auth/register	Public	Create user, resolve tenant, issue cookies
POST /api/auth/logout	Cookie	Invalidate the refresh jti, clear cookies
GET /api/auth/me	Session	Current user; falls back to JWT claims if the DB is down
POST /api/auth/refresh	Refresh cookie	Issue a new access token
GET /api/auth/access-status	Session	Which data sources this user can reach
GET /api/auth/setup	?secret=SETUP_SECRET	Create/upgrade the auth schema. 503 when the secret is unset

7.3 Admin (role-gated)

Route	Purpose
GET /api/admin/tenants	List DB tenants plus static TENANT_CONFIG and domain maps
POST /api/admin/tenants	Register or upsert a tenant and its domain mappings
PATCH /api/admin/tenants/[key]	Partial update of a DB-registered tenant
DELETE /api/admin/tenants/[key]	Soft-delete (deactivate)
POST /api/admin/tenants/probe	Heuristic auto-detect of a bridge kind from a URL

POST body: tenantKey*, displayName*, kind*, url*, ucids*, xTenant, the has* *capability flags*, coachActivityIds, authHeaderName, authHeaderValue, domains.

By design. The probe endpoint returns a **suggestion, not a decision**. Bridge kinds are inferred from response shape, which is ambiguous for some endpoints, so the wizard requires a human to confirm before saving.

7.4 Utility and diagnostics

Route	Auth	Purpose
POST /api/ai	Session	LLM Q&A over dashboard context
GET, POST /api/ai/[...path]	None	Reverse proxy to the FastAPI service; 502 when unreachable
GET /api/branding	Session	Branding settings and resolved client brand
PUT /api/branding	Session	Save branding (validated and normalized)
GET /api/health	None	Bridge connectivity and env diagnostics
GET /api/debug/banco-test	Session	Three raw bridge probes; not admin-gated
GET /api/debug/second-brain-check	None	Second Brain lookup by email or org

Security note. Three routes are reachable without a session: /api/health, /api/ai/[...path] and /api/debug/second-brain-check. The last two accept parameters and reach upstreams, so they should be gated before a public deployment. Tracked in section 14.

7.5 AI service (FastAPI)

Mounted at /ai by ai-service/app/main.py. Reached from the browser through the Next.js proxy at /api/ai/*. **No endpoint carries auth**, so the service must not be exposed publicly — the proxy is the intended entry point.

Method and path	Purpose	Request -> Response
GET /health	Health, llm_enabled, db_configured	-> dict
POST /ai/discover-company	Company discovery, no persist	CompanyIn -> CompanyKnowledge
POST /ai/discover-services	Company + service discovery, persists	CompanyIn -> CompanyKnowledge
POST /ai/generate-dashboard	Create a background job	GenerateRequest -> JobState
GET /ai/status/{job_id}	Poll job state (404 if unknown)	-> JobState
POST /ai/provide-ids	Resume a job paused at needs_ids (409 if not waiting)	ProvideIdsIn -> JobState
POST /ai/confirm-services	Resume a job paused at review_services	ConfirmServicesIn -> JobState
POST /ai/generate-sync	Whole pipeline in one request (serverless-friendly)	GenerateRequest -> JobState
POST /ai/publish	Publish a job's config; registers domain routing	PublishIn -> {published, slug}
GET /ai/dashboard/{slug}	Published config, metadata only	-> DashboardConfig
GET /ai/render/{slug}	Published config plus live widget preview	-> {config, preview}
DELETE /ai/knowledge/{slug}	Drop cached discovery so the next run re-probes	-> {cleared}

8. TypeScript domain contracts

These types in `lib/types.ts` are the contract every connector normalizes into and every page reads. Adding a client means writing an adapter to these types, not adding a branch to a page.

8.1 Envelope and filters

```
ApiResponse<T> { success: boolean; data: T; meta: ApiMeta }
ApiMeta       { filters: Record<string, unknown>; timestamp: string; source: string }

Module        'lms' | 'coach' | 'simulator' | 'certification' | 'second-brain'
DateRange     { from: Date; to: Date }
DashboardFilters { selectedModules: Module[]; dateRange: DateRange; customerId?: number }

KpiCard       { label, labelKey, value: number|string, delta: number,
               unit?: string, tier: 'A'|'B', noComparison?: boolean }
TimeSeriesPoint { date: string /* YYYY-MM-DD */; value: number; value2?: number }
```

8.2 Live API response shapes

Null is meaningful throughout: `avgScore: null` means the upstream reported no scores, which is different from an average of zero. Pages must render "no data" for null and a real zero for `0`.

```
OverviewApiResponse { totalEvaluations, avgScore|null, passRate|null,
                      passedEvaluations, prevTotalEvaluations,
                      prevAvgScore|null, prevPassRate|null }
TrendsApiResponse   { scoreTrend, passFailTrend, evalCountTrend,
                      scoreDistribution?: ScoreDistributionBucket[] }
EvaluationApiRow     { savedReportId, usecaseId|null, usecaseName|null,
                      score|null, result|null, passed, date }
UsecaseApiRow       { usecaseId, usecase_name|null, totalEvaluations,
                      avgScore|null, passRate|null, passed }
BestPerformerRow    { user_email, user_name|null, sessions, avg_score, pass_rate }
allTimeStats?      { totalRecords, avgBestScore, recordsGe80,
                      uniqueUsers, uniqueSims } -- hasTopStats only
ObjectionRow        { usecaseId, objectionText, count, passRate,
                      modelAnswer|null, topAnswers: {text, name}[] }
BusinessLineRow     { tagId, name, memberCount, simCount, avgScore|null, activeUsers }
OrganizationApiResponse { totalMembers, totalAdmins, totalSupervisors,
                          members: OrgMemberRow[], admins: OrgAdminRow[] }
```



```
LmsApiResponse      { configured: boolean, enrolledUsers, totalEnrollments,
                      totalCourses, modulesCompleted,
                      completionRate|null, avgQuizScore|null,
                      completionTrend: ApiTrendPoint[], courses: LmsCourseRow[] }
LmsCourseRow        { courseId, name, enrolled, completed,
                      completionRate|null, avgScore|null }
```

Why LMS has its own contract. `LmsApiResponse` is deliberately **not** shaped like `OverviewApiResponse`. An LMS measures course progress (enrolled, completed, quiz score); a Simulator measures scored practice sessions. Sharing one response type is what let Simulator figures render under an LMS heading, so the types are kept apart on purpose. `configured: false` is the signal to render an empty state.

8.3 Tenant and identity types

```

AuthUser      { id, email, full_name, customer_id, role: 'user'|'admin', created_at }
JwtClaims     { user_id, email, customer_id, jti, iat, exp }
AccessStatus  { hasCoachData, hasSecondBrainData, hasAnyAccess }

PharmaTenant  = string          -- open, not a literal union: wizard-registered
                                   -- tenants must be representable at runtime
TenantConfig  { kind: 'sale_exercises'|'kpi'|'exceltis_rest'
               url, xTenant?, ucids?: number[]
               hasCertification?, hasObjections?, hasBusinessLines?
               hasOrganization?, hasTopStats?, hasLms?, hasSimulator?
               coachActivityIds?: number[]
               authHeaderName?, authHeaderValue? }
PharmaTenantRow -- camelCase view of the pharma_tenants row, plus isActive/timestamps

```

PharmaTenant was narrowed to a literal union originally; it had to be widened to string so tenants registered at runtime through the admin wizard are expressible without a code change.

9. AI dashboard builder

The builder exists to make onboarding a data operation. It probes a client's upstream, works out which services and metrics exist, proposes a dashboard layout, validates every widget against live data, and publishes a config that the frontend renders generically. `ai-service/app/agents/` holds one agent per stage; `workflow.py` is the sequential runtime the API uses, mirrored by a formal LangGraph StateGraph in `graph.py`.

Degrades deliberately. The LLM is **optional**. With no anthropic key the agents fall back to heuristics, and `asyncpg` is optional too, so the builder runs with neither an LLM nor a database. That keeps onboarding usable in local development and stops the pipeline from being a hard dependency on model availability.

9.1 Job phases

```
queued -> planning -> company_discovery -> service_discovery
      |
      | (bridge cannot list its own IDs)
      v
      [needs_ids] <-- human input
      |
-> schema_discovery -> [review_services] <-- human confirmation
-> dashboard_planning -> dashboard_config -> validation
-> preview -> publish -> done                    (or -> error)
```

The two bracketed phases are **deliberate human-in-the-loop pauses**, not failures. `needs_ids` occurs because some bridges cannot enumerate their own exercise IDs, and guessing the allowlist would silently widen or narrow a client's scope. `review_services` asks a human to confirm the discovered module set before anything is published.

9.2 Core models

ServiceKind	pharma_kpi pharma_sale_exercises pharma_exceltis_rest coach_app_sql second_brain rolplay_app_sql unknown
MetricType	count score rate dimension timeseries table
WidgetType	kpi_tile line_chart bar_chart donut table histogram
GenerateRequest	company* (min_length 1), exercise_ids[], domains[], services[] -- contracted services, guided step 1 manager_request, auto_publish
CompanyKnowledge	company, slug, domains[], services[ServiceDescriptor], exercise_ids[], coach_activity_ids[], last_discovery, source: 'fresh' 'cache'
DiscoveredMetric	key, label, type, unit, source_kind, source_action, sample_value, supported, raw_field -- dotted path into real JSON
WidgetConfig	id, type, title, metric_key, dimension, metrics[], source_kind, source_action, span (1-4), raw_field
DashboardConfig	company, slug, title, connector, connector_handle, rows[DashboardRow], filters[], recommendations[], branding, version, created_at
ValidationReport	ok, issues[ValidationIssue], summary, has_errors
JobState	job_id, request, phase, percent, logs[], knowledge, schema_ (alias "schema"), dashboard, validation, preview, published, error, pending_connector, available_modules

`DiscoveredMetric.raw_field` is a dotted path into the real upstream JSON, so a published widget reads the field that was actually observed during discovery rather than a field name the model guessed.

9.3 Validation before publish

The validation and preview phases execute every proposed widget against live data. A widget whose metric does not resolve is reported as a `ValidationIssue` and `POST /ai/publish` returns 400 rather than shipping a dashboard with broken tiles. This is the automated form of the "never invent data" rule: a widget must prove it can render before it exists.

9.4 The guided configuration flow

The operator-facing flow at /dashboard-builder is three sections of one page — no wizard framework, no separate route.

1. **Contracted services.** Multi-select cards (Simulator, Master Coach, Certifier Coach, LMS, Second Brain), all selected by default so a rushed operator cannot create an empty dashboard. Sent as `services[]`.
2. **Identifier.** Company name is the only required field. The email domain is recommended (it routes logins) but can be derived from the client's users; exercise IDs and a Second Brain admin email sit behind **Advanced** for the exceptions.
3. **Fetch, review, publish.** Each discovered service is listed with its real counts, plus honest "no data yet" and "not contracted" lines, and a preview of the layout. Publishing registers domain routing so the client's users can sign in immediately.

The rule that matters. Step 1 captures **intent**; the platform independently verifies which of those services have data. A service that is contracted but empty is hidden rather than shown as zeros, and a service with data that was not ticked stays off. Contracted AND has-data is the condition for rendering.

Full wireframes: `docs/guided-config-wireframe.md`.

10. Frontend structure

Next.js App Router. `app/layout.tsx` composes providers in a fixed order: `ThemeProvider` -> `HtmlLangSync` -> `AuthProvider` -> `ClientBrandProvider` -> `LayoutContent`. Branding must load after auth because it is tenant-scoped.

10.1 Routes

Route	Shows	Gating
/	Landing page, or the global overview when signed in	Auth state
/simulator	Simulator KPIs, score trend, per-usecase table	Module
/coach	Master Coach: usecases, best performers, coaching insights	Module
/certification	Certifier Coach: pass/fail stacked bar, results	Module
/lms	LMS: enrollments, completion rate, quiz scores, per-course table	Module
/second-brain	Members, message logs, documents	Module
/activities	Per-activity list with score colouring	Access caps
/conversational	Objection cards, expandable	hasPharmaAccess
/business-lines	Business-line metrics	Pharma + capability
/organization	Members, admins, roles	Pharma + capability
/reports	Multi-solution CSV export and report templates	Session
/drilldown/[id]	One evaluation in full, on-demand ES/EN translation, CSV	Session
/settings	Branding: logo, colours, theme presets, platform name	Session
/admin/tenants	Client-onboarding wizard	Admin
/dashboard-builder	AI builder: guided config and job progress	Session
/d/[slug]	A published AI-generated dashboard	Session
/auth/login, /auth/register	Authentication	Public
/privacy, /terms	Legal	Public

10.2 Dynamic rendering

No page is hardcoded per client. Two mechanisms make the whole dashboard adapt: capability-driven navigation, and config-driven widget rendering.

- `useAvailableModules()` reads `/api/dashboard/modules`; `Sidebar.tsx` filters its entries from that. While loading it assumes **all** modules, so navigation never flickers items away.
- `DashboardRenderer.tsx` renders an AI-generated `DashboardConfig` — rows of widgets with a type and a span — so a published dashboard needs no bespoke page component.
- `useSnapDateRange()` snaps the default window once to the tenant's real data bounds from `/api/dashboard/data-bounds`, so a new client does not open on an empty chart.

10.3 Client state and data

<code>lib/store.ts</code>	<code>zustand</code> : <code>selectedModules</code> , <code>selectedSolution</code> , <code>dateRange</code> , <code>rangeInitialized</code> , <code>refreshKey</code> . Default window 24 months — a 30-day default was hiding real history.
<code>useApi()</code>	Generic fetch hook. Unwraps <code>{success, data, meta}</code> , 35 s timeout, <code>credentials: 'include'</code> .
<code>buildApiUrl()</code>	Builds range-scoped URLs; <code>rk: refreshKey</code> busts the cache.
Charts	<code>recharts</code> wrappers: <code>ActivityLineChart</code> , <code>DonutChart</code> , <code>ModuleBarChart</code> , <code>StackedBarChart</code> .
Primitives	<code>SummaryCard</code> , <code>MetricCard</code> , <code>ChartCard</code> , <code>DataTable</code> , <code>EmptyState</code> , <code>ExportButton</code> , <code>DateRangePicker</code> , plus <code>shadcn ui</code> .

11. Localization and demo mode

11.1 Localization

A static dictionary, no i18n library. `lib/translations.ts` holds flat `en` and `es` objects; `lib/lang-store.ts` is a zustand store persisted to `localStorage['rp-lang']`, **defaulting to Spanish**, exposing `useT()` which components consume as `const t = useT(); t.someKey`.

- `components/HtmlLangSync.tsx` keeps `<html lang>` in sync so Chrome's auto-translate does not rewrite the brand name "Rolplay" to "Roleplay".
- `lib/hooks/useTranslation.ts` is a **different**, AI-backed runtime translator for free-text content that comes from upstreams and cannot be pre-translated. It batches `N` strings into one call and caches in `sessionStorage`.
- `lib/field-labels.ts` and `lib/field-map.ts` normalize upstream field names into human labels, since each bridge names its columns differently.

11.2 Demo mode

Demo mode exists for sales demonstrations, and its central risk is a real client seeing synthetic numbers. Scoping is therefore narrow and tested.

Property	Behaviour
Activation	<code>NEXT_PUBLIC_DEMO_MODE === 'true'</code> , or the signed-in email is on a Rolplay-owned domain (<code>rolplay.ai app net com</code> , extendable via <code>DEMO_DOMAINS</code>)
Single check	<code>useDemoData(email)</code> — called in each route before <code>resolveOrgType()</code> , so a demo session never reaches a real upstream
Deterministic	Seeded PRNG (mulberry32) keyed off the date range plus a per-solution salt — no flicker between reloads
Range-aware	Figures scale with the selected window, so changing the range behaves like real data
Self-consistent	<code>passedEvaluations = total × passRate</code> ; LMS completions sum to the per-course totals
Module-honest	<code>demoLms()</code> produces course metrics, not relabelled session counts — the same rule real pipelines follow
Guarded	<code>lib/__tests__/demo-scope.test.ts</code> asserts a real client email never activates demo mode

Runbook: `docs/DEMO_RUNBOOK.md`.

12. Technology stack

12.1 Next.js application

Concern	Choice	Version
Framework	Next.js, App Router	16.2.2
UI runtime	React	19.2.4
Language	TypeScript	5.x
Client state	zustand	5.0.12
Styling	Tailwind CSS (@tailwindcss/postcss), shadcn 4, @base-ui/react 1.3	4.x
Charts	recharts	3.8.1
Animation / icons	framer-motion 12, lucide-react 1.7	—
Dates	date-fns	4.x
Postgres driver	pg	8.20
MySQL driver	mysql2 (fallback / local direct mode)	3.22
Auth	jose (used by lib/jwt.ts), bcryptjs 3	—
Branding	colorthief (logo palette extraction)	—
Tests	vitest + jsdom, @testing-library/react 16, @vitest/coverage-v8	4.1.5
Lint	ESLint 9 + eslint-config-next	9.x

Read before coding. AGENTS.md records that this Next.js major has breaking changes relative to older conventions. Read the relevant guide under node_modules/next/dist/docs/ before writing new routing or data-fetching code rather than relying on recalled Next.js patterns.

Test suite

67 tests. vitest.config.ts sets the @ alias and excludes .claude/worktrees/ and the self-referential dashboard/ directory, which would otherwise be collected twice.

- lib/__tests__/org-type.test.ts — the tenant classifier, the highest-risk function for cross-tenant leaks.
- lib/__tests__/bridge-banco-analytics.test.ts — the closingretro score parsing and 0-100 normalization.
- lib/__tests__/demo-scope.test.ts — asserts real client emails never activate demo mode.
- lib/__tests__/module-gating.test.ts — capability gating, the anti-duplication rule.
- lib/__tests__/trend-transform.test.ts — time-series bucketing.
- components/__tests__/ — DashboardContent, Sidebar rendering.

12.2 Python AI service

Concern	Choice	Version
API	FastAPI + uvicorn	0.115.6 / 0.34
Models	pydantic, pydantic-settings	2.10.4 / 2.7.1
HTTP client	httpx	0.28.1
Orchestration	langgraph	0.2.60
LLM	anthropic (optional — heuristic fallback)	0.42.0
Postgres	asyncpg (optional — in-memory fallback)	0.30

Entrypoints: app/main.py (the mounted service), api/index.py, and standalone/index.py — a reduced single-file variant (/health, POST /ai/generate-sync, GET /) for serverless targets. The standalone variant is not mounted by app/main.py.

13. Deployment and configuration

13.1 Environment variables

Grouped by subsystem. Never read a server-only variable from a client component — only `NEXT_PUBLIC_*` is available in the browser.

Variable	Purpose
<code>AUTH_DATABASE_URL</code>	Postgres auth DB (identity, tenant config, AI tables)
<code>JWT_SECRET</code> , <code>REFRESH_SECRET</code>	Access (8 h) and refresh (7 d) token signing
<code>SETUP_SECRET</code>	Gates <code>GET /api/auth/setup</code> ; 503 when unset
<code>BRIDGE_URL</code> , <code>BRIDGE_SECRET</code>	PHP SQL bridge endpoint and X-Bridge-Key
<code>DB_HOST</code> , <code>DB_USER</code> , <code>DB_PASSWORD</code> , <code>DB_NAME</code>	Direct MySQL, local development only
<code>PHARMA_BRIDGE_BASE_URL</code>	Enables the Sanfer / Apotex static tenant entries
<code>PHARMA_TENANT_DOMAINS</code>	"tenant:domain, ..." static domain map (DB rows win)
<code>BANCO_EMAIL_DOMAINS</code>	Domains routed to the Banco pipeline
<code>ROLPLAY_APP_SQL_URL</code> , <code>ROLPLAY_APP_LOGINS</code>	rolplay.app SQL endpoint and login map
<code>SECOND_BRAIN_API_URL</code> , <code>SECOND_BRAIN_API_TOKEN</code>	Second Brain REST API
<code>SB_OWNER_OVERRIDES</code>	Owner-email overrides where the <code>admin@domain</code> convention fails
<code>LMS_API_URL</code>	LearnWorlds school base URL, e.g. <code>https://<school>.learnworlds.com</code>
<code>LMS_CLIENT_ID</code> , <code>LMS_CLIENT_SECRET</code>	OAuth2 client credentials; <code>Lw-Client</code> header
<code>LMS_ACCESS_TOKEN</code>	Optional static token; skips the credentials exchange
<code>LMS_<TENANT>_*</code>	Per-tenant override of any of the four above (e.g. <code>LMS_APOTEX_API_URL</code>)
<code>AI_SERVICE_URL</code>	FastAPI service, default <code>http://127.0.0.1:8088</code>
<code>NEXT_PUBLIC_DEMO_MODE</code> , <code>DEMO_DOMAINS</code>	Demo mode activation and extra demo domains
<code>DYNAMIC_TENANTS_TTL_MS</code>	Dynamic tenant cache TTL, default 30 000 ms

Secrets. Credentials belong in `.env.local` (git-ignored) or the platform's secret store — never in `.env.local.example`, which is committed and must contain placeholders only.

13.2 Targets and bootstrap

Next.js app	Vercel (<code>vercel.json</code>), Render (<code>render.yaml</code>), or Docker (<code>docker-compose.yaml</code>).
AI service	Its own Dockerfile, <code>render.yaml</code> , <code>vercel.json</code> . Must not be publicly exposed — it has no auth; reach it through <code>/api/ai/*</code> .
PHP bridge	<code>rolplay-bridge.php</code> deployed alongside the MySQL upstreams. See <code>MODIFY_PHP_BRIDGE.md</code> .
Schema bootstrap	<code>GET /api/auth/setup?secret=...</code> creates and upgrades the auth schema idempotently (<code>CREATE TABLE IF NOT EXISTS</code>).
Migrations	Numbered files in <code>migrations/</code> run against the auth Postgres (001 is MySQL and reflects an earlier design). 004 and 005 must be applied for branding keys and <code>rolplay.app</code> routing.

13.3 Onboarding a client

Three supported routes, in increasing order of automation:

1. **Admin wizard** (/admin/tenants) — register the tenant key, bridge kind, URL, ucids, capability flags and login domains. Takes effect immediately: the write invalidates the dynamic tenant cache.
2. **AI builder** (/dashboard-builder) — guided configuration, discovery, validation, publish. Produces a rendered dashboard at /d/[slug].
3. **Static config** — add an entry to TENANT_CONFIG plus a PHARMA_TENANT_DOMAINS mapping. Requires a deploy; used for the hand-verified built-ins.

Runbooks: docs/ONBOARDING_A_CLIENT.md, docs/NEW_USER_ONBOARDING.md.

14. Known issues and open items

Everything below was verified against the code at the commit on the cover page. It is recorded here rather than omitted, because each item is a real trap for the next person.

14.1 Verified discrepancies

#	Issue	Detail and impact
1	Two users shapes	Migration 002 declares <code>company_id VARCHAR(100)</code> ; the live bootstrap in <code>app/api/auth/setup/route.ts</code> declares <code>customer_id INTEGER NOT NULL DEFAULT 0</code> . All TypeScript uses <code>customer_id</code> — the setup route is authoritative and migration 002 reflects the older design. Trust the setup route when reconciling a database by hand.
2	Two <code>saved_reports</code> shapes	<code>coach_app</code> has <code>score</code> , <code>passed_flag</code> , <code>coach_user_id</code> ; <code>rolplay_pro</code> has <code>banco_user_id</code> and no score columns (scores are parsed from <code>closingretro</code> HTML). Same table name, different meaning — check which database a query targets before reusing SQL.
3	Undocumented tables	<code>branding_settings</code> and <code>tenant_integrations</code> have DDL only inside the setup route, with no numbered migration. A database provisioned purely from <code>migrations/</code> will lack them.
4	Legacy branding key	<code>branding_settings.customer_id</code> retains its <code>UNIQUE</code> constraint even though <code>tenant_key</code> (migration 004) is now the real key. Two tenants that both resolve to <code>customer_id = 0</code> can collide on insert.
5	Unauthenticated routes	<code>/api/ai/[...path]</code> and <code>/api/debug/second-brain-check</code> take parameters and reach upstreams without a session. <code>/api/debug/banco-test</code> requires a session but is not admin-gated. Gate all three before a public deployment.
6	Stale AI prompt	<code>lib/ai.ts</code> (a separate Gemini 2.5 Flash path, distinct from <code>ai-service/</code>) still carries a leftover "TCF French learning assistant" system prompt instead of a dashboard prompt.
7	Duplicate trees	<code>dashboard/</code> and <code>.claude/worktrees/*</code> duplicate the main tree and are excluded from <code>vitest</code> . Edits there do not ship.

14.2 Open items

Item	Status	What is needed
LMS capability flags in the DB	In progress	<code>hasLms</code> and <code>hasSimulator</code> exist on <code>TenantConfig</code> but have no <code>has_lms</code> / <code>has_simulator</code> columns yet. Needs migration 006, the <code>db-tenants.ts</code> mapping, the admin API fields, and the builder selector, so the flags are settable from the UI rather than only in code.
LearnWorlds end-to-end verification	Blocked	The connector is written against the documented LearnWorlds v2 API but has not been run against a live school. The response mapping (field names, progress scale, pagination) must be confirmed with real credentials in <code>.env.local</code> before the numbers can be trusted.
Value KPIs (approval rate, learning curve)	Blocked	Needs a definition for "approved divided by active training period": per-week velocity, per-day, or percentage of active users.
Tenant isolation test with two rival logins	Pending	Procedure is written in <code>docs/tenant-isolation.md</code> ; needs a human to run it with two real client accounts.
Metric normalization ratification	Pending	The written recommendation is in <code>docs/tenant-isolation.md</code> ; needs sign-off so score and pass-rate definitions are identical across clients.
Clients not yet onboarded	Blocked	<code>lacosteAsistentes</code> has no confirmed email domain. Gentera and Salinas identify users by employee ID, not email, so domain-based resolution cannot work as-is. <code>BancoPPEL</code> and <code>Lily</code> run separate Flask containers, not yet wired.

Current LMS behaviour. The LMS pipeline was added specifically to stop Simulator data rendering under an LMS label. Until item 2 above is verified against a live LearnWorlds school, `/api/dashboard/lms` returns `configured: false` for tenants without credentials, and the page shows an empty state — which is the correct behaviour, not a bug.

14.3 Where to look first

Wrong numbers for one client	Read <code>meta.source</code> on the API response, then that connector. Check the capability flags.
A module is missing from the nav	<code>GET /api/dashboard/modules</code> . Remember it fails open to the full set on error.

A client sees no data at all	Check <code>customer_id</code> in the JWT (0 means unlinked) and the domain map priority: DB, then env, then built-in aliases.
Cross-tenant suspicion	<code>lib/org-type.ts</code> and <code>resolvePharmaTenant()</code> . Run <code>lib/__tests__/org-type.test.ts</code> .
Demo data in production	<code>useDemoData()</code> and <code>DEMO_DOMAINS</code> . Run <code>demo-scope.test.ts</code> .